Internship Project Report

Push Notification API

Producer-Consumer System

A Flask-based distributed notification delivery system built on a Producer-Consumer architecture. Supports web, mobile, and pager device types with real-time SSE updates, a background worker pool, and SQLite persistence.

Submitted by: Aryan Lanke

Internship Organization: CDAC

Mentor: Praveen Dewangan And Sanjay Bhargava

Table of Contents

1. Project Overview
2. How to Run the Project
3. Architecture Overview
4. System Workflow
5. Block Diagram
6. API Endpoints
7. Communication Mechanism
8. Current Implementation
9. Limitations
10. Production Improvements
11. Hosting, Deployment & Integration Guide
12. Future Scope / Enhancements
13. Technologies Used

1. Project Overview

What This API Does

This project is a Push Notification Delivery API. It is not a standalone end-user application; rather, it is a robust backend API designed to be integrated into other systems (like your company's banking app, e-commerce backend, or IoT monitoring system).

It acts as your own private notification microservice. When your main backend decides a user needs to be notified (e.g., "Transaction successful"), it simply pings this API, and this system handles the heavy lifting of delivering that message to the user's web browser, mobile device, or pager.

What Problem It Solves

In modern applications (banking apps, e-commerce platforms, healthcare systems), events happen on the server side (e.g., "Transaction approved", "Order shipped") and users need to be notified instantly on their devices.

Without a dedicated system:

- Browsers and apps have to constantly ask the server "Are there new messages?" — this is wasteful.
- Sending to hundreds of devices one-by-one is slow and blocks the main application.
- There is no central record of what was sent, to whom, and whether it succeeded.

This system solves all three problems:

- **SSE (Server-Sent Events)** pushes notifications to browsers instantly — no polling.
- **A background worker pool** processes delivery to multiple devices in parallel.
- **SQLite** stores every notification job with its status and delivery results.

2. How to Run the Project

Prerequisites

- Python 3.8+
- pip package manager
- (Optional) Docker & Docker Compose

Option A — Run Locally (Without Docker)

Step 1 — Clone the Repository

```
git clone https://github.com/AryanLanke/Push-Notification-Api.git
cd Push-Notification-Api
```

Step 2 — Set Up the Producer

```
cd producer
pip install -r requirements.txt
python producer.py
```

The Producer API starts at: **http://localhost:5000**

Step 3 — Set Up the Consumer (in a new terminal)

```
cd consumer
pip install -r requirements.txt
python consumer.py
```

The Consumer dashboard starts at: **http://localhost:5001**

The Consumer automatically registers itself with the Producer on startup.

Step 4 — Send a Test Notification

Open the Producer dashboard at http://localhost:5000 and use the built-in UI.

Consumer Command-Line Options

You can run multiple consumer instances on different ports:

```
# Default (port 5001)
python consumer.py

# Custom port and name
python consumer.py --port 5002 --name "Device-2"

# Point to a different producer
python consumer.py --port 5003 --producer-port 5000 --name "Device-3"
```

Option B — Run with Docker Compose

```
docker-compose up --build
```

Service	URL
Producer	http://localhost:5000
Consumer	http://localhost:5001

To stop:

```
docker-compose down
```

Ports Reference

Service	Default Port	Description
Producer API	5000	REST API + Admin Dashboard
Consumer App	5001	Notification receiver + Dashboard
Consumer #2	5002	(optional) Second consumer device

3. Architecture Overview

This system is divided into two independent services:

Producer (Port 5000)

The central notification **server**. It:

- Accepts incoming notification requests from external applications via a REST API.
- Stores device registrations in a SQLite database.
- Puts notification jobs into an in-memory queue.
- Runs a pool of 5 background worker threads that pick jobs from the queue and deliver them.
- Pushes real-time status updates to the admin dashboard via SSE.

Consumer (Port 5001)

Represents a **client device** (web, mobile, or pager). It:

- Auto-registers itself with the Producer when it starts.
- Exposes a /receive endpoint that the Producer's workers call to deliver notifications.
- Pushes received notifications to the browser instantly using SSE — no page refresh needed.

- Can be run as multiple independent instances on different ports to simulate multiple devices.

Producer — Modular Code Structure

The Producer is split into four files for clean separation of concerns:

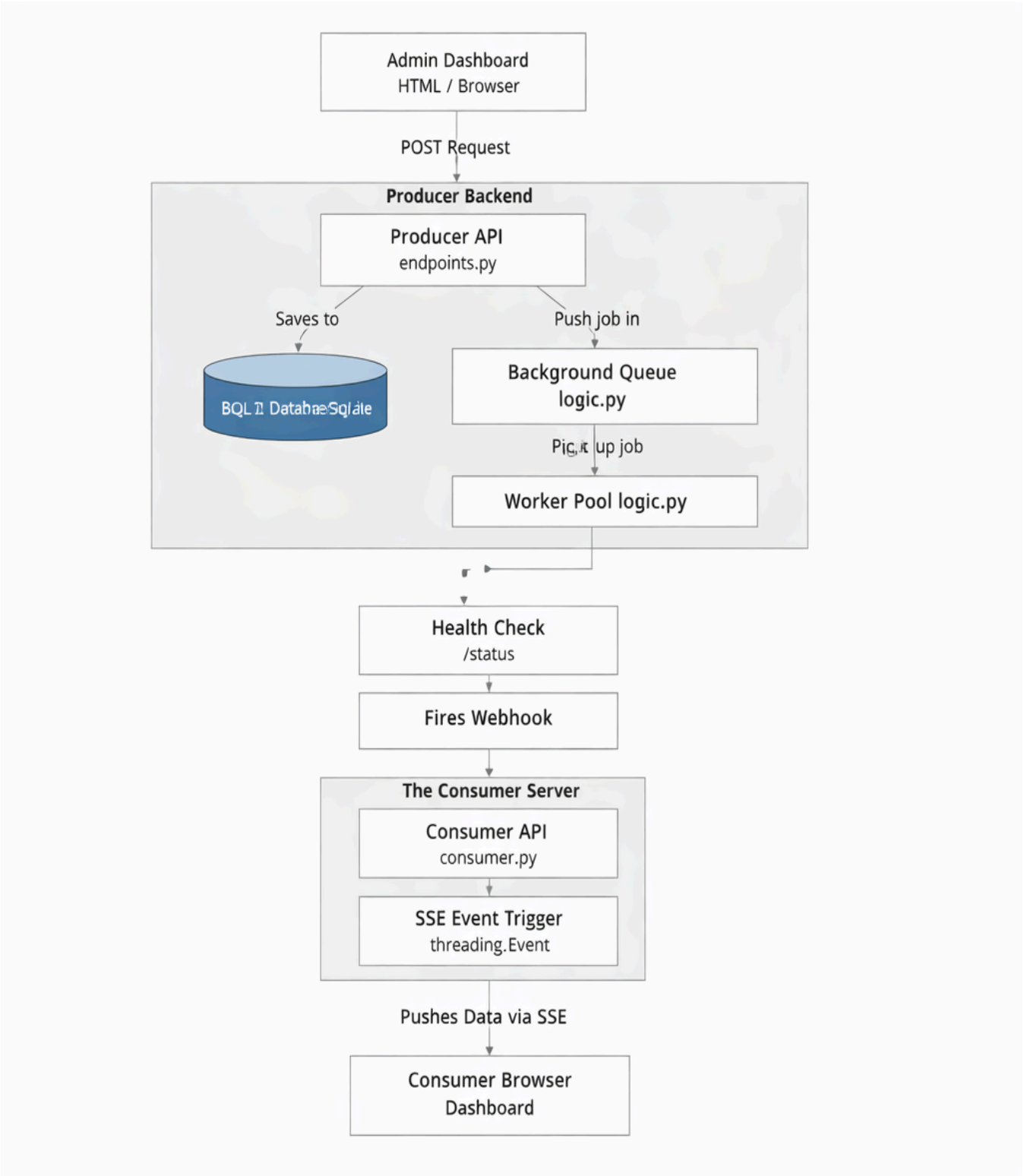
File	Responsibility
producer.py	App factory, configuration, error handlers, startup
database.py	SQLAlchemy database models (Device, Notification)
endpoints.py	All REST API endpoints (Blueprint)
logic.py	Worker pool, queue, notification handlers (web/mobile/pager), SSE

4. System Workflow

Step-by-step flow from a notification being requested to it being received:

```
1. [External App / Admin UI]
   → Sends POST /notifications/send with { title, message }
2. [Producer — endpoints.py]
   → Validates the request
   → Looks up registered devices in SQLite
   → Creates a Notification record (status = "queued")
   → Packages a "job" and puts it in the in-memory Queue
   → Immediately responds 202 Accepted + job_id
3. [Producer — logic.py (Background Worker)]
   → One of 5 worker threads picks the job from the Queue
   → Loops through each target device
   → Spawns a mini-thread per device for parallel delivery
4. [Delivery — based on device type]
   → Web device (with VAPID sub): sends real browser push via Web Push Protocol
   → Web device (IP/port): sends HTTP POST to Consumer's /receive endpoint
   → Mobile device (FCM token): sends via Google Firebase Cloud Messaging
   → Mobile device (simulated): logs success (Firebase not configured)
   → Pager device: simulated alert (pager gateway not configured)
5. [Consumer — consumer.py]
   → /receive endpoint accepts the notification
   → Stores it in memory
   → Signals the SSE event (rings the bell)
6. [Consumer Browser]
   → SSE stream receives the signal instantly
   → Browser UI updates without any page reload
7. [Producer — logic.py]
   → Worker records final results (success/failure per device)
   → Updates Notification in SQLite (status = "completed")
   → Broadcasts job_complete event via SSE to Producer dashboard
```

5. Block Diagram



6. API Endpoints

Producer Endpoints (Base URL: http://localhost:5000)

Method	Endpoint	Description
GET	/	Admin dashboard (web UI)
GET	/api	API discovery — lists all endpoints
GET	/events	SSE stream for real-time dashboard updates
GET	/devices	List all registered devices
POST	/devices/register	Register a new consumer device
DELETE	/devices/<device_id>	Unregister a device
POST	/notifications/send	Enqueue a notification for delivery
GET	/notifications/queue	View jobs currently queued or processing
GET	/notifications/status/<job_id>	Check status of a specific notification job
GET	/notifications/history	View last 50 sent notifications
GET	/vapid/public-key	Get VAPID public key for web push subscription

Example: Register a Device

```
POST /devices/register
Content-Type: application/json

{
  "name": "My Laptop Browser",
  "device_type": "web",
  "ip_address": "127.0.0.1",
  "port": 5001
}
```

Response (201 Created):

```
{
  "success": true,
  "message": "Device registered successfully",
```

```
  "device": {
    "device_id": "abc123",
    "name": "My Laptop Browser",
    "device_type": "web",
    "ip_address": "127.0.0.1",
    "port": 5001
  }
}
```

Example: Send a Notification

```
POST /notifications/send
Content-Type: application/json

{
  "title": "Transaction Alert",
  "message": "Your payment of ₹500 was successful.",
  "device_ids": ["abc123"]  # Optional: omit to broadcast to ALL devices
}
```

Response (202 Accepted):

```
{
  "success": true,
  "job_id": "f4a1c2d3-...",
  "status": "queued",
  "total_devices": 1,
  "hint": "Track status at GET /notifications/status/f4a1c2d3-..."
}
```

Consumer Endpoints (Base URL: http://localhost:5001)

Method	Endpoint	Description
GET	/	Consumer dashboard showing received notifications
POST	/receive	Called by the Producer to deliver a notification
GET	/events	SSE stream — pushes new notifications to the browser
GET	/status	Health check — returns device info and counts

7. Communication Mechanism

This system uses two communication techniques:

HTTP (REST API)

Standard request-response communication. Used for:

- Registering devices
- Sending notifications
- Checking job status
- Fetching history

This is the standard way web applications talk to each other. The sender makes a request, the server responds immediately.

SSE — Server-Sent Events

SSE is a technique where the server pushes data to the browser over a long-lived HTTP connection — without the browser having to ask repeatedly.

```
Browser                          Server
   |                                |
   |-- GET /events ---------------► |  (browser opens a long connection)
   |                                |
   |                                |  (new notification arrives at /receive)
   |◄-- data: {"title":"..."} ----- |  (server immediately pushes it)
   |                                |
   |◄-- : keep-alive -------------- |  (every 30s, keep the connection alive)
```

Why SSE instead of polling?

Polling (Old Way)	SSE (This System)
Browser asks every 5 seconds	Server pushes instantly
Wasteful — many empty checks	Efficient — one lasting connection
Adds latency (up to 5s delay)	Near-zero latency
More server load	Less server load

SSE is used in two places in this system:

Producer Dashboard — gets real-time job status updates (queued → processing → completed).**How it works in this system:**

Consumer Dashboard — gets new notifications the instant they arrive.

8. Current Implementation

This is a working prototype designed to demonstrate the Producer-Consumer architecture and core notification delivery pipeline.

What Is Real

- ✅ Full REST API with proper validation and error handling
- ✅ Device registration and management (SQLite)
- ✅ Asynchronous job queue with a 5-thread worker pool
- ✅ Parallel delivery to multiple devices simultaneously
- ✅ Real-time SSE updates on both Producer and Consumer dashboards
- ✅ Web Push via VAPID protocol (real browser notifications, if VAPID keys configured)
- ✅ Firebase FCM integration code (real mobile push, if Firebase credentials configured)
- ✅ Job status tracking (queued → processing → completed)
- ✅ Notification history stored in SQLite
- ✅ Docker Compose setup for running both services together
- ✅ Automated test suites for Producer and Consumer

What Is Simulated

- 📱 **Mobile (FCM)** — Simulated with a time.sleep(0.2) when Firebase credentials are not configured.
- 📟 **Pager alerts** — Simulated with a time.sleep(0.1) (no real pager gateway integrated).
- 🌐 **Web Push** — Falls back to HTTP POST to Consumer's /receive endpoint when no VAPID subscription data is provided.

To enable real delivery:

- Web Push: Generate VAPID keys and add them to producer/.env
- Mobile FCM: Create a Firebase project and add FIREBASE_CREDENTIALS_FILE to producer/.env

9. Limitations

Understanding the limitations helps you know when and how to upgrade this system for production use.

Limitation	Details
SQLite database	File-based, single-process. Not suitable for distributed deployments or high write loads.

In-memory queue	Uses Python's queue.Queue. Jobs are lost if the Producer crashes or restarts.
No authentication	Any caller can register devices, send notifications, or view history. No API keys or JWT.
Consumer must be active	If the Consumer is offline when a notification is sent, the delivery fails permanently. No retry logic.
Single Producer instance	One Flask process handles all requests. Cannot scale horizontally without a message broker.
No rate limiting	The API accepts unlimited requests — vulnerable to flooding in an open network.
In-memory SSE clients	SSE connections are stored per process — will break in a multi-process deployment.

10. Production Improvements

While this system functions well as a working prototype, to deploy it in a real production environment, your company should make the following improvements:

Database (Persistence)

Current: SQLite (File-based, single-process)

Replace with: PostgreSQL

What it will solve: Handles thousands of concurrent connections, supports replication, prevents database locking issues, and works dynamically across multiple distributed servers.

In-Memory Message Queue

Current: Python queue.Queue (RAM-based)

Replace with: Redis (via Celery/RQ) or Apache Kafka

What it will solve: Currently, if the server restarts, all queued notifications in memory are permanently lost. Redis/Kafka ensures jobs are persisted outside of the application process and can be distributed safely across multiple worker nodes.

Push Services (Mobile & Pager)

Current: Simulated push events (e.g., time.sleep()).

Replace with: Real FCM (Firebase Cloud Messaging) for mobile and a real Pager gateway (like PagerDuty or Twilio).

What it will solve: Delivers real OS-level push notifications to mobile devices even when the app is fully closed, rather than just simulating a successful delivery to the API.

Security (Authentication & Authorization)

Current: Open, unauthenticated API endpoints.

Replace with: API Keys (for systems triggering notifications) and JWTs (for consumer devices).

What it will solve: Ensures only authorized microservices in your company can trigger push alerts and prevents malicious users from flooding the system with fake registration or notification requests.

Fault Tolerance (Retry Mechanisms)

Current: Fire-and-forget (Failures are logged, but not retried).

Replace with: Exponential back-off retries and Dead-Letter Queues (DLQ).

What it will solve: If a consumer device or third-party service (like Apple APNs) is temporarily down, the system will automatically try sending the notification again later instead of giving up permanently.

Scale (Load Balancing)

Current: Single Flask server process.

Replace with: Nginx (Reverse Proxy/Load Balancer) managing multiple server instances.

What it will solve: Distributes high-traffic incoming API requests efficiently. If one instance crashes, the load balancer reroutes traffic to healthy instances, ensuring zero downtime.

11. Hosting, Deployment & Integration Guide

Phase 1: Code Modifications (What to Change)

Before hosting this system in production, update the placeholder components:

1. Change the Database: Open .env and producer.py. Replace the SQLite database URI with a connection string to a hosted PostgreSQL database (e.g., AWS RDS).

2. Setup Real Mobile Push: In logic.py (send_mobile_push), provide real Google Firebase Credentials so it stops simulating and starts sending actual mobile nudges.

3. Upgrade the Queue: Replace the local python queue.Queue in logic.py with a connection to a Redis server for high-availability setups.

Phase 2: Hosting the API (Where to Put It)

This API operates as an independent microservice and must run continuously on a backend infrastructure.

Custom Domain Deployment: Companies will typically host this on their own cloud infrastructure (such as AWS EC2 or an ECS cluster via Docker) and map it to a custom subdomain (e.g., api.notifications.yourcompany.com).

The provided Dockerfile and docker-compose.yml can be utilized directly to run containerized deployments.

Phase 3: Integration Examples & Use Cases

Any application can interact with this API using standard HTTP POST requests. SDKs are not strictly required.

Example 1: Banking / Finance App

Event: A customer's transaction is processed on the backend.

Action: The main server sends POST /notifications/send with { "title": "Transaction Alert", "message": "₹5,000 debited" }.

Result: Registered devices receive the secure alert instantly.

Example 2: E-Commerce Order System

Event: Order status updates to "Shipped".

Action: The order service system sends POST /notifications/send targeting the customer's specific linked device_id.

Result: Customer's phone immediately shows "Your order has been shipped!".

Example 3: IoT / Monitoring System

Event: A temperature sensor exceeds safe limits.

Action: IoT hub sends POST /notifications/send with the alert details.

Result: The on-call engineer's mobile device and desktop dashboard receive the high-priority alert.

Integration Steps

Step 1. Frontend Team (Registering Users)

Whenever a user logs into your company's app or website, the frontend code makes an HTTP POST request to register that device:

```
POST https://api.notifications.yourcompany.com/devices/register
{
  "name": "User's iPhone",
  "device_type": "mobile",
  "subscription_data": "<their-fcm-token>"
}
```

Step 2. Backend Team (Sending Alerts)

Whenever the system needs to alert a user, send the payload to the API:

```
POST https://api.notifications.yourcompany.com/notifications/send
{
  "title": "Payment Received",
  "message": "We got your $50.00!",
  "device_ids": ["<device-id-from-step-1>"]
}
```

The API accepts the payload, creates a job, and delegates delivery to the worker pool.

12. Future Scope / Enhancements

🌐 Real Push Notifications (FCM + Web Push)

Fully configure Firebase Cloud Messaging for Android/iOS apps and VAPID Web Push for browsers. This enables notifications even when the browser or app is closed — the OS delivers them natively.

🔌 WebSockets for Bidirectional Communication

The current SSE implementation is one-directional (server → browser). Upgrading to WebSockets (via flask-socketio or a dedicated WebSocket server) enables bidirectional communication — useful for chat, live updates, and acknowledgement reporting.

📊 Monitoring & Logging

- Integrate Prometheus + Grafana for metrics dashboards (queue depth, delivery rates, error rates).
- Add structured logging with ELK Stack (Elasticsearch, Logstash, Kibana) or Loki + Grafana.
- Set up Sentry for real-time error tracking and alerting.

🔁 Retry Mechanisms

- Implement exponential back-off for failed deliveries.
- Add a dead-letter queue (DLQ) where permanently failed notifications are stored for manual review or replay.
- Track delivery attempts and max retries per notification.

💾 Message Persistence & Durability

- Replace in-memory queue with Redis Streams or Kafka for guaranteed message delivery — even if the Producer restarts, jobs are not lost.
- Store notification payloads with TTL (time-to-live) for offline consumers.

🌍 Multi-Region Deployment

- Deploy Producer instances in multiple cloud regions (US, EU, Asia) to reduce latency for geographically distributed consumers.
- Use a global load balancer (AWS Global Accelerator, Cloudflare) to route requests to the nearest region.
- Replicate the message broker across regions for failover.

🔐 Authentication & Security

- API key management for client systems.
- JWT-based authentication for consumer devices.
- Role-based access control (RBAC): admin, sender, viewer roles.
- End-to-end encryption for notification payloads.

📱 Native Mobile SDK

Build lightweight SDKs for Android (Kotlin/Java) and iOS (Swift) that handle device registration, FCM token management, and notification display automatically.

13. Technologies Used

Category	Technology	Usage
Backend	Python 3.8+	Core language
Web Framework	Flask	REST API and SSE endpoints
Database ORM	Flask-SQLAlchemy	Database models and queries
Database	SQLite	Device registration and notification history
Queue	Python queue.Queue	In-memory notification job queue
Concurrency	Python threading	Worker pool (5 threads) + per-device dispatch
Push Delivery	pywebpush + VAPID	Real browser push notifications
Mobile Push	Firebase Admin SDK (FCM)	Real mobile push (when configured)
HTTP Client	requests	Producer → Consumer HTTP delivery calls
CORS	Flask-CORS	Cross-origin request support
Environment	python-dotenv	Secure key management via .env
Containerization	Docker + Docker Compose	Packaging and local multi-service deployment
Testing	pytest	Automated unit tests for Producer & Consumer
Real-time	SSE (Server-Sent Events)	Live dashboard updates (no polling)

📁 Project Structure

```
Push-Notification-Api/
│
├── producer/                   # Producer Service (Port 5000)
│   ├── producer.py             # App factory, config, startup
│   ├── models.py               # SQLAlchemy models (Device, Notification)
│   ├── routes.py               # All REST API endpoints
│   ├── services.py             # Worker pool, queue, push handlers
│   ├── templates/
│   │   └── producer.html       # Admin dashboard UI
│   ├── requirements.txt        # Python dependencies
│   ├── Dockerfile              # Container definition
│   └── test_producer.py        # Pytest test suite
│
├── consumer/                   # Consumer Service (Port 5001)
│   ├── consumer.py             # Consumer app (receives & displays notifications)
│   ├── templates/
│   │   └── consumer.html       # Consumer dashboard UI
│   ├── requirements.txt        # Python dependencies
│   ├── Dockerfile              # Container definition
│   └── test_consumer.py        # Pytest test suite
│
├── docker-compose.yml          # Runs both services together
├── .gitignore
└── README.md                   # This file
```

Built as an internship project to demonstrate Producer-Consumer architecture, real-time notification delivery, and distributed system design patterns using Python and Flask.